

Ingeniería del Software II

Práctica #4 – Fuzzing

Ejercicio 1

Sea el siguiente programa:

```
1 def crashme(s: str) -> None:
2     if len(s) > 0 and s[0] == 'b':
3         if len(s) > 1 and s[1] == 'a':
4             if len(s) > 2 and s[2] == 'd':
5                 if len(s) > 3 and s[3] == '!':
6                     raise Exception()
```

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=5$ y el siguiente conjunto inicial de inputs: ' ' (cadena vacía), 'a', 'b', 'c', 'd', 'aa', 'ab', 'ac', 'ba', 'bb', 'bc', 'bad'.

- Indicar el camino de ejecución de cada input
- Indicar la energía de cada input en el conjunto inicial.
- ¿Cuál es la probabilidad que el fuzzer elija el input "bad" para mutarlo?

Ejercicio 2

Sea el siguiente programa:

```
1 def target_function(input_string: str) -> int:
2     tokens = input_string.split()
3     stack = []
4     for token in tokens:
5         if token.isdigit():
6             stack.append(int(token))
7         else:
8             try:
9                 arg2 = stack.pop()
10                arg1 = stack.pop()
11                if token == "+":
12                    result = arg1 + arg2
13                elif token == "-":
14                    result = arg1 - arg2
15                elif token == "*":
16                    result = arg1 * arg2
17                elif token == "/":
18                    result = arg1 / arg2
19                else:
20                    raise ValueError("Invalid operator: %s" % token)
21                stack.append(result)
22            except:
23                raise ValueError("Invalid expression")
24    return stack[0]
```

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=3$. Sea el siguiente conjunto inicial de inputs: '1', '2', '3', '1 2 +', '2 1 +', '1 1 +', '1 2 /', '2 1 /', '1 3 *', '2 4 *', '2 4 *' y '1 2 + 4 *'.

- Indicar el camino de ejecución de cada input
- Indicar la energía de cada input en el conjunto inicial.

c) ¿Cuál es la probabilidad que el fuzzer elija el input '1 2 + 4 *' para mutarlo?

Ejercicio 3

Sea el siguiente programa `cgi_decode`

```

1 def cgi_decode(s: str) -> str:
2     # Mapping of hex digits to their integer values
3     hex_values = {
4         '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
5         '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
6         'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
7         'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
8     }
9
10    t = ""
11    i = 0
12    while i < len(s): // c1
13        c = s[i]
14        if c == '+': // c2
15            t += '_'
16        elif c == '%': // c3
17            digit_high, digit_low = s[i + 1], s[i + 2]
18            i += 2
19            if digit_high in hex_values and digit_low in hex_values: // c4 and c5
20                v = hex_values[digit_high] * 16 + hex_values[digit_low]
21                t += chr(v)
22            else:
23                raise ValueError("Invalid encoding")
24        else:
25            t += c
26            i += 1
27    return t

```

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=4$. Sea el siguiente conjunto inicial de inputs: "hola+mundo", "hello+world", "gracias+mundo", "%12", "%AA", "%BB", "mistring".

a) Indicar la energía de cada input en el conjunto inicial.

b) ¿Cuál es la probabilidad que el fuzzer elija el input "mistring" para mutarlo?

Ejercicio 4

Sea el siguiente programa:

```

1 def parse_url_manual(url: str) -> [str]:
2     scheme = ""
3     netloc = ""
4     path = ""
5     params = ""
6     query = ""
7     fragment = ""
8
9     # Split the URL into parts
10    if "://" in url:
11        scheme, url = url.split("://", 1)
12    if "/" in url:
13        netloc, url = url.split("/", 1)
14    if "?" in url:
15        path, query = url.split("?", 1)
16    elif "#" in url:
17        path, fragment = url.split("#", 1)

```

```

18     else:
19         path = url
20
21     # Split query into parameters
22     query_params = {}
23     if query:
24         for param in query.split("&"):
25             key, value = param.split("=") if "=" in param else (param, "")
26             query_params[key] = value
27
28     return [scheme, netloc, path, params, query, fragment]

```

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=3$. Sea el siguiente conjunto inicial de inputs:

- `https://www.example.com/path/to/resource`
- `http://example.com/page?p1=v1&p2=v2`
- `ftp://ftp.example.com/resource#section1`
- `https://www.example.com/path?p1=v1`
- `https://www.example.com/path?p2=v2`
- `https://www.example.com/path?p3=v3`

¿Cuál es la probabilidad que el fuzzer elija el input `"https://www.example.com/path?p3=v3"` para mutar?

Ejercicio 5

Sea el siguiente programa:

```

1 def parse_email(email: str) -> List[Optional[str]]:
2     parts = email.split('@')
3     if len(parts) != 2:
4         return [None, None, None]
5     else:
6         local_part, domain = parts
7         if not local_part:
8             return [None, None, None]
9         else:
10            if not domain or '.' not in domain:
11                return [None, None, None]
12            else:
13                last_dot_index = domain.rfind('.')
14                domain_part = domain[:last_dot_index]
15                top_level_domain = domain[last_dot_index + 1:]
16                return [local_part, domain_part, top_level_domain]

```

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=3$. Sea el siguiente conjunto inicial de inputs:

- #1: `"user@example.com"`
- #2 `"john.doe@example.co.uk"`
- #3 `"alice.smith@subdomain.example.org"`
- #4 `"info@company.com"`
- #5 `"support@sub.subdomain.example.net"`
- #6 `"invalid-email"`
- #7 `"missing_domain@"`
- #8 `"@missing_local_part.com"`
- #9 `"missing_local_part_and_domain"`
- #10 `"user@[IPv6:2001:db8::1]"`
- #11 `"user+tag@example.com"`

- #12 ‘‘john.smith@subdomain.example.org’’

- a. Por cada input en el conjunto inicial, indicar su camino de ejecución, y su energía asignada luego de ejecutar todo el conjunto inicial.
- b. ¿Cuál es la probabilidad que el fuzzer elija el input ‘‘alice.smith@subdomain.example.org’’ para mutar?

Ejercicio 6

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=3$. Sea el siguiente programa y conjunto inicial de inputs:

```

1 def parse_file_path(file_path: str) -> List[Optional[str]]:
2     if '/' not in file_path and '\\' not in file_path: #L2
3         return [None, None, None] #L3
4     elif '/' in file_path: #L4
5         last_separator_index = file_path.rfind('/') #L5
6     else:
7         last_separator_index = file_path.rfind('\\') #L7
8
9     directory = file_path[:last_separator_index] #L9
10    filename = file_path[last_separator_index + 1:] #L10
11
12    if not filename: #L12
13        return [directory, None, None] #L13
14
15    last_dot_index = filename.rfind('.') #L15
16    if last_dot_index != -1: #L16
17        name = filename[:last_dot_index] #L17
18        extension = filename[last_dot_index + 1:] #L18
19    else:
20        name = filename #L20
21        extension = None #L21
22
23    return [directory, name, extension] #L23

```

- #1 ‘‘example’’
- #2 ‘‘example.txt’’
- #3 ‘‘’’
- #4 ‘‘filename’’
- #5 ‘‘/home/user/documents/’’
- #6 ‘‘/’’
- #7: ‘‘/home/user/documents/report.pdf’’
- #8 ‘‘/home/user/documents/report’’

- a. Por cada input en el conjunto inicial, indicar su camino de ejecución, y su energía asignada luego de ejecutar todo el conjunto inicial.
- b. ¿Cuál es la probabilidad que el fuzzer elija el input #7 ‘‘/home/user/documents/report.pdf’’ para mutar?

Ejercicio 7

Asumiendo que tenemos un boosted greybox fuzzer con exponente $a=3$. Sea el siguiente programa y conjunto inicial de inputs:

```

1 def parse_log_entry(log_entry: str) -> List[Optional[str]]:
2     if log_entry.startswith("[") and "]" in log_entry: #L2
3         timestamp_end = log_entry.find("]") #L3
4         timestamp = log_entry[1:timestamp_end] #L4
5         log_part = log_entry[timestamp_end + len("]_"):] #L5
6         log_level_end = log_part.find(":") #L6
7         if log_level_end != -1: #L7
8             log_level = log_part[:log_level_end] #L8
9             user_start = log_part.find("User_") + len("User_") #L9
10            if user_start != len("User_") - 1: #L10
11                user_end = log_part.find("'", user_start) #L11
12                if user_end != -1: #L12
13                    user = log_part[user_start:user_end] #L13
14                    ip_start_marker = "logged_in_from_" #L14
15                    ip_start = log_part.find(ip_start_marker) + len(ip_start_marker) #L15
16                    if ip_start != len(ip_start_marker) - 1: #L16
17                        ip_address = log_part[ip_start:].strip() #L17
18                        return [timestamp, log_level, user, ip_address] #L18
19
20 return None #L20

```

- #1 "[2024-11-25 15:28:00] INFO: "
- #2: "[2024-11-25 15:29:00] INFO: User 'admin' logged"
- #3 "[2024-11-25 15:30:00] INFO: User 'admin' logged in from "
- #4 "[2024-11-25 15:31:00] INFO: User 'student0' logged in from "
- #5 "[2024-11-25 15:32:00] INFO: User 'student2' logged in from "
- #6 "[2024-11-25 15:32:00] INFO: User 'student3' logged in from "
- #7 "[2024-11-25 15:32:00] INFO: User 'student4' logged in from "
- #8 "[2024-11-25 15:30:00] INFO: User 'student4' logged in from 192.168.1.1"

- a. Por cada input en el conjunto inicial, indicar su camino de ejecución, y su energía asignada luego de ejecutar todo el conjunto inicial.
- b. ¿Cuál es la probabilidad que el fuzzer elija el input #8 "[2024-11-25 15:30:00] INFO: User 'student4' logged in from 192.168.1.1" para mutar?